

MLOps — Diabetes MLflow Project

Entregables — Tema 4: MLflow Models & Registry
Máster en Inteligencia Artificial · Loyola Másteres

Este documento recoge los entregables del proyecto de clasificación de diabetes usando MLflow. El objetivo es detectar casos de diabetes (clasificación binaria) priorizando el recall como métrica principal, dado que en un contexto médico un falso negativo (no detectar un caso real) es más peligroso que un falso positivo. El dataset utilizado es el Pima Indians Diabetes Dataset, con 768 muestras y 8 features médicas.

1. Tabla de modelos registrados

1.1 Criterio de registro

De las 72 combinaciones del grid search, solo se registran en el MLflow Model Registry aquellos modelos que cumplen simultáneamente los siguientes criterios sobre el set de validación:

- **Recall ≥ 0.70** — criterio médico principal: detectar el máximo de diabéticos reales.
- **F1 ≥ 0.65** — garantía de equilibrio mínimo entre recall y precisión.

Los modelos que no superan estos umbrales se descartan porque, aunque puedan tener recall alto (como threshold=0.3 con recall=1.0), tienen una precision tan baja que generarían demasiadas falsas alarmas, siendo inútiles en la práctica clínica. Los modelos con threshold=0.5 y solver=lbfgs tampoco se registran porque su recall no supera el mínimo de 0.70 exigido.

1.2 Modelos registrados (selección representativa)

La siguiente tabla muestra una selección representativa de los modelos registrados, ordenados por f1 descendente. Se incluyen los más relevantes para ilustrar el efecto de cada hiperparámetro:

C	Solver	max_iter	Threshold	Recall	Precision	F1
0.1	liblinear	100	0.4	0.7963	0.6935	0.7414
0.1	liblinear	500	0.4	0.7963	0.6935	0.7414
0.1	liblinear	1000	0.4	0.7963	0.6935	0.7414
1.0	lbfgs	100	0.4	0.7593	0.6949	0.7257
1.0	liblinear	100	0.4	0.7593	0.6949	0.7257

C	Solver	max_iter	Threshold	Recall	Precision	F1
10.0	liblinear	500	0.4	0.7593	0.6949	0.7257
0.1	lbfgs	100	0.4	0.7222	0.6964	0.7091
0.1	lbfgs	100	0.3	0.8519	0.6133	0.7132
0.01	liblinear	100	0.4	0.8519	0.6053	0.7077
0.01	lbfgs	100	0.4	0.7037	0.6786	0.6909
0.01	liblinear	100	0.3	0.8889	0.5581	0.6857

* Fila en verde: modelo seleccionado y promovido a Production. Las versiones con max_iter=500 y max_iter=1000 para C=0.1/liblinear/thr=0.4 producen idénticas métricas, confirmando que max_iter no es relevante para este dataset.

1.3 Modelo ganador seleccionado

C=0.1, solver=liblinear, max_iter=100, threshold=0.4 — recall=0.7963 | precision=0.6935 | f1=0.7414

Se elige este modelo porque ofrece el mejor equilibrio entre recall y precisión entre todos los registrados. Frente a las alternativas con threshold=0.3 (que alcanzan recall=0.85-0.89 pero con precisión de 0.55-0.61), este modelo detecta el 80% de los diabéticos reales manteniendo una precisión del 69%, lo que lo hace más útil en un contexto clínico real donde las falsas alarmas también tienen un coste.

2. Script de evaluación sobre test set

El script **evaluate.py** evalúa el modelo en Production sobre el 10% del dataset reservado para test (77 muestras), enviando las peticiones a la API de MLflow. El threshold de 0.4 se aplica automáticamente porque viaja embebido dentro del modelo gracias al wrapper `ThresholdClassifier`.

```
import requests
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from dataset import load_data, split_data

API_URL = "http://127.0.0.1:6000/invocations"

def predict_proba_api(data: pd.DataFrame) -> list:
    """Llama a la API y obtiene las predicciones (threshold embebido en el modelo)."""
    payload = {"dataframe_split": {
        "columns": data.columns.tolist(),
        "data": data.values.tolist()
    }}
    response = requests.post(API_URL, json=payload)
    response.raise_for_status()
    return response.json()["predictions"]

def evaluate_test():
    X, y = load_data()
    _, _, X_test, _, _, y_test = split_data(X, y)

    print(f"\nEvaluando sobre {len(X_test)} muestras de test...")

    # La API aplica el threshold internamente (ThresholdClassifier con thr=0.4)
    preds = predict_proba_api(X_test)
    y_pred = preds

    metrics = {
        "accuracy": round(accuracy_score(y_test, y_pred), 4),
        "precision": round(precision_score(y_test, y_pred), 4),
        "recall": round(recall_score(y_test, y_pred), 4),
        "f1": round(f1_score(y_test, y_pred), 4),
    }

    print(f"\n{'█'*40}")
    print(f" METRICAS REALES SOBRE TEST (10%)")
    print(f"{'█'*40}")
    for k, v in metrics.items():
        print(f" {k:<12}: {v}")
    print(f"{'█'*40}")

    return metrics

if __name__ == "__main__":
```

```
evaluate_test()
```

Salida del script:

Train:	537 muestras (70%)
Validacion:	154 muestras (20%)
Test:	77 muestras (10%)

Evaluando sobre 77 muestras de test...

MÉTRICAS REALES SOBRE TEST (10%)

```
accuracy : 0.7143
precision : 0.5806
recall : 0.6667
f1 : 0.6207
```

Comparativa validación vs test:

Métrica	Validación	Test
accuracy	~0.76	0.7143
precision	0.6935	0.5806
recall	0.7963	0.6667
f1	0.7414	0.6207

3. Conclusiones

3.1 Hallazgos del grid search

- **max_iter no tiene impacto relevante.** Las métricas son idénticas para max_iter=100, 500 y 1000 con el mismo resto de parámetros. Esto indica que el modelo converge en pocas iteraciones con este dataset, y aumentar max_iter solo incrementa el tiempo de entrenamiento sin beneficio.
- **C alto no mejora las métricas.** A partir de C=0.1 las métricas se estabilizan. C=10.0 produce resultados prácticamente idénticos a C=1.0, lo que sugiere que la regularización moderada (C=0.1) es suficiente y previene sobreajuste.
- **El threshold es la palanca clave.** Reducir el threshold de 0.5 a 0.4 mejora significativamente el recall (de ~0.63 a ~0.80 con liblinear/C=0.1) a costa de reducir algo la precisión. Reducirlo más a 0.3 eleva el recall hasta 0.85-1.0 pero la precisión cae por debajo de 0.60, lo que fue descartado por generar demasiadas falsas alarmas.
- **liblinear supera a lbfgs en recall con threshold=0.4.** Con los mismos parámetros C y max_iter, liblinear consistentemente alcanza recall=0.7963 frente al 0.7222 de lbfgs. Esto se debe a diferencias en el algoritmo de optimización que afectan a la frontera de decisión en datos con clases desbalanceadas (268 positivos vs 500 negativos en este dataset).

3.2 Validación vs test

Se observa una caída de métricas entre validación y test (recall: 0.7963 → 0.6667). Esta caída no indica necesariamente overfitting del modelo, sino que es esperable dada la pequeña dimensión del conjunto de test (77 muestras, ~27 positivos). Con tan pocos positivos, cada predicción errónea tiene un impacto porcentual alto. El modelo sigue cumpliendo el criterio mínimo de recall ≥ 0.65 en test.

3.3 Valor de MLflow en este proyecto

El uso de MLflow ha permitido registrar, comparar y reproducir los 72 experimentos de forma sistemática. El Model Registry ha facilitado la promoción del modelo ganador a Production y su servicio inmediato via API sin cambiar código. La integración del threshold dentro del propio modelo (ThresholdClassifier) garantiza consistencia entre entrenamiento y servicio, eliminando un error habitual en proyectos de ML donde el threshold se aplica manualmente fuera del modelo.